Lecture 15: Spark SQL and DataFrames

DSL351 / DSP352: Big Data Analytics

Amit Kumar Dhar

ED1, 406A, amitkdhar@iitbhilai

March 12, 2026

Outline

Introduction

Class 15 Topics Overview

Focus: Advanced Operations and External Data Sources in Spark SQL.

- **User-Defined Functions (UDFs):** Creation and performance impact.
- **Querying Tools:** Spark SQL Shell, Beeline, and Tableau.
- **External Data Sources:** Interacting with JDBC, PostgreSQL, MySQL, Azure Cosmos DB, MS SQL Server.
- **Higher-Order Functions:** Manipulating complex data types (Arrays, Maps).
- **Common DataFrame Operations:** Grouping, Aggregations, Window Functions, Unions, and Joins.

User-Defined Functions (UDFs)

Introduction to UDFs

While Apache Spark provides many built-in functions, the flexibility of Spark allows for defining custom functions.

- These custom functions are known as **User-Defined Functions (UDFs)**.
- UDFs are crucial for data engineers and scientists to encapsulate complex logic.
- E.g., Wrapping a Machine Learning model within a UDF so a data analyst can query predictions via Spark SQL.

Spark SQL UDFs

- Creating PySpark or Scala UDFs makes them usable within Spark SQL queries.
- **Note:** UDFs operate per session and will not be persisted in the underlying metastore.

Registering UDFs (Scala)

```scala
// In Scala
// Create cubed function
val cubed = (s: Long) => {
  s * s * s
}

// Register UDF
spark.udf.register("cubed", cubed)

// Create temporary view
spark.range(1, 9).createOrReplaceTempView("udf_test")
```

Scala UDF Example

Registering UDFs (Python)

```python
# In Python
from pyspark.sql.types import LongType

# Create cubed function
def cubed(s):
    return s * s * s

# Register UDF
spark.udf.register("cubed", cubed, LongType())

# Generate temporary view
spark.range(1, 9).createOrReplaceTempView("udf_test")
```

Python UDF Example

Using UDFs in Spark SQL

Once registered, the UDF can be used in any Spark SQL query within the session:

```
// In Scala/Python
// Query the cubed UDF
spark.sql("SELECT id, cubed(id) AS id_cubed FROM udf_test").show()

/* Output:
+---+---------+
| id|id_cubed|
+---+---------+
|  1|        1|
|  2|        8|
|  3|       27|
...
*/
```

Querying the UDF

Evaluation Order in Spark SQL

Spark SQL (DataFrame API, Dataset API) does **not guarantee** the order of evaluation of subexpressions.

```
1 spark.sql("SELECT s FROM test1 WHERE s IS NOT NULL AND strlen(s) > 1")
```

Unpredictable Evaluation

The clause `s is NOT NULL` is not guaranteed to execute prior to `strlen(s) > 1`.

Proper Null Checking in UDFs

To handle potential nulls effectively without relying on evaluation order:

1. Make the UDF itself null-aware and do null checking *inside* the UDF.
2. Use `IF` or `CASE WHEN` expressions to do the null check and invoke the UDF in a conditional branch.

Speeding up PySpark UDFs

Issue: PySpark UDFs historically had slower performance than Scala UDFs due to expensive data movement between the JVM and Python.

Solution: Pandas UDFs (Vectorized UDFs).

- Introduced in Spark 2.3.
- Uses **Apache Arrow** to transfer data efficiently.
- Operates on a Pandas Series or DataFrame instead of individual row-by-row inputs.

Pandas UDF Categories

From Apache Spark 3.0, Pandas UDFs split into two API categories:

- **Pandas UDFs:** Infer the type from Python type hints (`pandas.Series`, `pandas.DataFrame`, `Tuple`, `Iterator`).
- **Pandas Function APIs:** Directly apply a local Python function to a PySpark DataFrame where input/output are Pandas instances (e.g., `map`, `map_partitions`).

Pandas UDF Example (Python)

```python
# In Python
import pandas as pd
from pyspark.sql.functions import col, pandas_udf
from pyspark.sql.types import LongType

# Declare the cubed function
def cubed(a: pd.Series) -> pd.Series:
    return a * a * a

# Create the pandas UDF for the cubed function
cubed_udf = pandas_udf(cubed, returnType=LongType())
```

Declaring a Pandas UDF

Pandas UDF Vectorized Execution

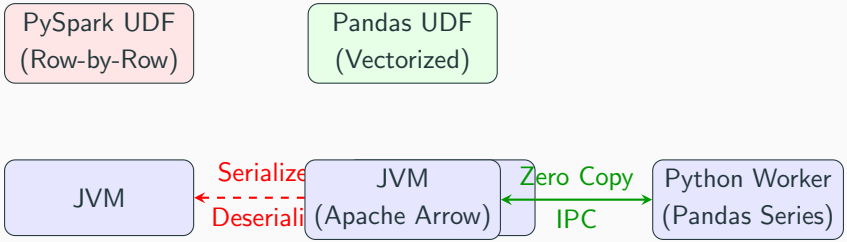
```
# Create a Spark DataFrame
df = spark.range(1, 4)

# Execute function as a Spark vectorized UDF
df.select("id", cubed_udf(col("id"))).show()

/* Output:
+---+----------+
| id|cubed(id)|
+---+----------+
|  1|         1|
|  2|         8|
|  3|        27|
+---+----------+
*/
```

Executing the Pandas UDF

Performance: PySpark UDF vs Pandas UDF



Querying Spark SQL

Querying with Spark SQL Shell

- `spark-sql` CLI is a convenient tool for executing Spark SQL queries.
- Communicates with the Hive metastore service in local mode.
- Start it via: `./bin/spark-sql`

Spark SQL Shell: Create Table

```
spark-sql> CREATE TABLE people (name STRING, age int);

20/01/11 22:42:16 WARN HiveMetaStore: Location: file:/user/hive/warehouse/people
specified for non-external table:people
Time taken: 0.63 seconds
```

Creating a Permanent Table

Spark SQL Shell: Insert and Query

```
spark-sql> INSERT INTO people VALUES ("Michael", NULL);
spark-sql> INSERT INTO people VALUES ("Andy", 30);
spark-sql> INSERT INTO people VALUES ("Samantha", 19);

spark-sql> SELECT * FROM people WHERE age < 20;
Samantha    19
Time taken: 0.593 seconds, Fetched 1 row(s)
```

Inserting and Querying

Working with Beeline

- **Beeline** is a JDBC client utility for executing queries against the **Spark Thrift JDBC/ODBC server (STS)**.
- Allows JDBC/ODBC clients to execute SQL queries on Spark.

Starting the Thrift Server

```
# Ensure driver and worker are started
./sbin/start-all.sh

# Start Thrift Server
./sbin/start-thriftserver.sh
```

Starting STS

Connecting and Querying with Beeline

```
./bin/beeline

!connect jdbc:hive2://localhost:10000

0: jdbc:hive2://localhost:10000> SHOW tables;
+-----------+------------+--------------+
| database  | tableName  | isTemporary  |
+-----------+------------+--------------+
| default   | people     | false        |
+-----------+------------+--------------+
```

Beeline Usage

Working with Tableau

- You can connect BI tools like Tableau to Spark SQL via the Thrift JDBC/ODBC server.
- Requires Tableau's Spark ODBC driver.
- Connect via `localhost:10000` using the SparkThriftServer connector.

External Data Sources

External Data Sources: JDBC and SQL Databases

- Spark SQL includes a data source API to read from databases using JDBC.
- Returns results as a DataFrame.
- Driver JAR must be on the Spark classpath:

```
./bin/spark-shell --driver-class-path database.jar --jars database.jar
```

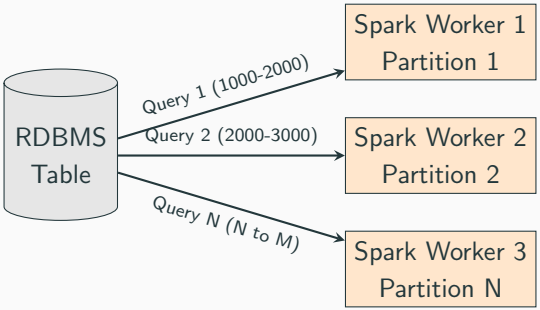
Common JDBC Connection Properties

- `url`: JDBC connection URL (e.g., `jdbc:postgresql://localhost...`)
- `dbtable`: Table to read/write.
- `query`: Query to fetch data (mutually exclusive with `dbtable`).
- `driver`: Class name of the JDBC driver.
- `user`, `password`: Credentials.

The Importance of Partitioning

- Without partitioning, all data transfers through **one driver connection**.
- This can saturate and slow down extraction.
- Parallelize reads using partitioning properties.

Partitioning Data Extraction



Partitioning Connection Properties

- `numPartitions`: Maximum number of partitions (and concurrent JDBC connections).
- `partitionColumn`: Numeric, date, or timestamp column used for striding.
- `lowerBound`: Minimum value of partition stride.
- `upperBound`: Maximum value of partition stride.

Partitioning Example

Settings: `numPartitions: 10`, `lowerBound: 1000`, `upperBound: 10000`

Translates to 10 queries, e.g.:

```
SELECT * FROM table WHERE partitionColumn BETWEEN 1000 and 2000;
SELECT * FROM table WHERE partitionColumn BETWEEN 2000 and 3000;
...
```

Partitioning Best Practices

- `numPartitions`: Use a multiple of the number of Spark workers.
- `lowerBound`/`upperBound`: Calculate based on actual minimum and maximum values to avoid empty partitions.
- `partitionColumn`: Choose a column that can be uniformly distributed to avoid data skew.

PostgreSQL Connection (Scala)

```scala
// In Scala
val jdbcDF1 = spark
  .read
  .format("jdbc")
  .option("url", "jdbc:postgresql:[DBSERVER]")
  .option("dbtable", "[SCHEMA].[TABLENAME]")
  .option("user", "[USERNAME]")
  .option("password", "[PASSWORD]")
  .load()
```

Loading Data from PostgreSQL

PostgreSQL Connection (Python)

```python
# In Python
jdbcDF1 = (spark
  .read
  .format("jdbc")
  .option("url", "jdbc:postgresql://[DBSERVER]")
  .option("dbtable", "[SCHEMA].[TABLENAME]")
  .option("user", "[USERNAME]")
  .option("password", "[PASSWORD]")
  .load())
```

Loading Data from PostgreSQL

MySQL Connection Example

Requires MySQL connector JAR on classpath.

```scala
// In Scala
val jdbcDF = spark
  .read
  .format("jdbc")
  .option("url", "jdbc:mysql://[DBSERVER]:3306/[DATABASE]")
  .option("driver", "com.mysql.jdbc.Driver")
  .option("dbtable", "[TABLENAME]")
  .option("user", "[USERNAME]")
  .option("password", "[PASSWORD]")
  .load()
```

Loading from MySQL

Azure Cosmos DB Integration

- Requires the `azure-cosmosdb-spark` connector.
- Uses `readConfig` dictionary mapping connection settings (Endpoint, Masterkey, Database, Collection).
- Supports custom queries via `query_custom` config property.
- MS SQL Server, Apache Cassandra, MongoDB, and Snowflake are also supported via external packages.

Higher-Order Functions and Complex Data Types

Manipulating Complex Data Types

Complex data types (Arrays, Maps) can be manipulated directly. Typical solutions:

1. Exploding the nested structure into rows, applying functions, and re-creating it.
2. Building a User-Defined Function (UDF).
3. Using **Higher-Order Functions**.

Option 1: Explode and Collect

Creates a new row for each element in the array, processes it, then groups.

```sql
-- In SQL
SELECT id, collect_list(value + 1) AS values
FROM (
  SELECT id, EXPLODE(values) AS value
  FROM table
) x
GROUP BY id
```

Drawback: `GROUP BY` requires shuffle operations. High memory usage. Order is not guaranteed.

Option 2: User-Defined Function

Map over elements inside a custom function.

```
spark.sql("SELECT id, plusOneInt(values) AS values FROM table")
```

Drawback: Serialization and deserialization process between JVM and Python/Scala can be expensive.

Built-in Array Functions

Spark 2.4+ includes native array functions:

- `array_distinct()`: Removes duplicates.
- `array_intersect()`, `array_union()`, `array_except()`
- `array_join()`: Concatenates using a delimiter.
- `array_sort()`, `concat()`, `flatten()`, `reverse()`
- `array_max()`, `array_min()`

Built-in Map Functions

Native map functions:

- `map_form_arrays()`: Creates map from key/value arrays.
- `map_from_entries()`: Creates map from struct array.
- `map_concat()`: Union of maps.
- `element_at()`: Value of given key.
- `cardinality()`: Size of array or map.

Higher-Order Functions: Introduction

Higher-order functions take anonymous lambda functions as arguments to transparently map over arrays.

- `transform()`
- `filter()`
- `exists()`
- `reduce()`

Higher-Order Functions: `transform()`

Produces an array by applying a function to each element.

```
-- Calculate Fahrenheit from Celsius
spark.sql("""
SELECT celsius,
       transform(celsius, t -> ((t * 9) div 5) + 32) as fahrenheit
FROM tC
""").show()
```

Higher-Order Functions: `filter()`

Produces an array consisting only of elements where the Boolean function is true.

```
-- Filter temperatures > 38C
spark.sql("""
SELECT celsius,
       filter(celsius, t -> t > 38) as high
FROM tC
""").show()
```

Higher-Order Functions: `exists()`

Returns true if the Boolean function holds for any element in the input array.

```
-- Is there a temperature of 38C?
spark.sql("""
SELECT celsius,
       exists(celsius, t -> t = 38) as threshold
FROM tC
""").show()
```

Higher-Order Functions: `reduce()`

Reduces the elements of an array to a single value by merging into a buffer.

```
-- Calculate average temperature
spark.sql("""
SELECT celsius,
       reduce(
         celsius,
         0,
         (t, acc) -> t + acc,
         acc -> (acc div size(celsius) * 9 div 5) + 32
       ) as avgFahrenheit
FROM tC
""").show()
```

Common DataFrame Operations

Common DataFrame Operations

Spark SQL provides a wide range of DataFrame operations:

- Aggregate functions (`groupBy`, `sum`, `count`, `agg`)
- Collection & Datetime functions
- Sorting & String functions
- **Unions** and **Joins**
- **Window functions**

DataFrame Setup Example

```scala
// Scala setup
val airports = spark.read.option("header","true").csv(airportsPath)
val delays = spark.read.option("header","true").csv(delaysPath)
  .withColumn("delay", expr("CAST(delay as INT) as delay"))
```

Setup DataFrames

DataFrame Unions

Unioning two DataFrames with the same schema:

```
// Scala
val bar = delays.union(foo)
bar.createOrReplaceTempView("bar")

bar.filter(expr("origin == 'SEA' AND destination == 'SFO'")).show()
```

Union DataFrames

DataFrame Joins

- Default is an `inner` join.
- Other types: `cross`, `outer`, `full`, `left`, `right`, `left_semi`, `left_anti`.

Join Example (Scala & Python)

```scala
foo.join(
  airports.as('air),
  $"air.IATA" === $"origin"
).select("City", "State", "delay", "distance").show()
```

Scala Join

```python
foo.join(
  airports,
  airports.IATA == foo.origin
).select("City", "State", "delay", "distance").show()
```

Python Join

Window Functions

Windowing: Introduction

A **window function** uses values from rows in a window (range of input rows) to return a set of values.

- Operates on a group of rows while returning a *single value for every input row*.
- Ideal for running totals, moving averages, and rankings.

Types of Window Functions

Type	Functions
Ranking	`rank()`, `dense_rank()`, `percent_rank()`, `ntile()`, `row_number()`
Analytic	`cume_dist()`, `first_value()`, `last_value()`, `lag()`, `lead()`

Window Function Example: Setup

First, aggregate total delays by origin and destination:

```
CREATE TABLE departureDelaysWindow AS
SELECT origin, destination, SUM(delay) AS TotalDelays
FROM departureDelays
WHERE origin IN ('SEA', 'SFO', 'JFK')
GROUP BY origin, destination;
```

Window Function Example: dense_rank()

Find the top 3 destinations with the most delays for each origin:

```
spark.sql("""
SELECT origin, destination, TotalDelays, rank
  FROM (
    SELECT origin, destination, TotalDelays, dense_rank()
      OVER (PARTITION BY origin ORDER BY TotalDelays DESC) as rank
    FROM departureDelaysWindow
  ) t
  WHERE rank <= 3
""").show()
```

DataFrame Modifications

Modifications: Adding Columns

DataFrames are immutable. Operations create new DataFrames.

```python
from pyspark.sql.functions import expr

foo2 = foo.withColumn(
    "status",
    expr("CASE WHEN delay <= 10 THEN 'On-time' ELSE 'Delayed' END")
)
foo2.show()
```

Adding a Column in Python

Modifications: Dropping & Renaming

```
val foo3 = foo2.drop("delay")
foo3.show()
```

Dropping a Column

```
foo4 = foo3.withColumnRenamed("status", "flight_status")
foo4.show()
```

Renaming a Column

Modifications: Pivoting

Swap columns for rows using `PIVOT`.

```
SELECT * FROM (
  SELECT destination, CAST(SUBSTRING(date, 0, 2) AS int) AS month, delay
  FROM departureDelays WHERE origin = 'SEA'
)
PIVOT (
  CAST(AVG(delay) AS DECIMAL(4, 2)) AS AvgDelay, MAX(delay) AS MaxDelay
  FOR month IN (1 JAN, 2 FEB)
)
ORDER BY destination
```

Summary

- **UDFs and Pandas UDFs** allow complex custom processing within Spark SQL.
- Spark can interact with diverse **External Data Sources** efficiently through JDBC, leveraging connection partitioning.
- **Higher-Order Functions** offer powerful, native techniques for nested and complex structures.
- Core relational operators: **Joins, Unions, and Window Functions** form the backbone of Spark data processing.

Questions?